MSSQL, Redis, HTTP and Gateway Performance Review

Summary

Performance changes must not weaken financial correctness. Operational tuning is configuration-driven where appropriate; accounting and locking invariants are not relaxed merely for benchmark numbers.

MSSQL connection pooling

SqlClient pooling is enabled through the connection string. Development baseline:

- Pooling=True;
- Min Pool Size=5;
- Max Pool Size=100;
- bounded connect timeout;
- Load Balance Timeout;
- application name.

SqlConnectionFactory creates short-lived logical connections; disposal returns physical connections to the pool. Production pool size should be tuned from measured database capacity.

Financial-transaction performance

Wallet transfer commits source/destination balances, idempotency, FinancialTransaction and Ledger in the same transaction. Lock duration matters, but not more than correctness. Wallet rows are locked in deterministic GUID order to reduce opposite-direction deadlock risk.

Isolation is not weakened solely to improve throughput.

Index approach

Existing unique/index structures support important hot paths. Speculative indexes on write-heavy financial tables may increase insert/update/log cost. New indexes should be driven by Query Store, execution plans, logical reads and measured p95/p99 evidence.

Monitor:

- pool exhaustion;
- login/transfer DB p95/p99;
- deadlocks;
- lock waits;
- top logical reads;
- Query Store regressions;
- transaction-log growth;
- index usage/fragmentation.

Redis

A single ConnectionMultiplexer remains singleton. Creating a multiplexer per request would be a socket/performance regression.

Configurable values:

- AbortOnConnectFail;
- connect retry;
- connect timeout;
- sync timeout;
- keep-alive;
- client name.

OTP fail-closed security is not weakened for performance. Redis is not a financial authority.

Monitor reconnects, latency, timeouts, CPU, memory, evictions, network and Lua latency.

HttpClient

Provider clients use HttpClientFactory + SocketsHttpHandler with:

- pooled connection lifetime;
- pooled idle timeout;
- max connections/server;
- GZip/Deflate/Brotli;
- provider-specific timeout;
- cookies disabled.

Provider calls go through YARP Gateway. Financial POSTs are not automatically retried without endpoint-specific idempotency guarantees.

YARP

Clusters use PowerOfTwoChoices. Production destination replicas can be added through configuration. Performance controls include:

- load balancing;
- active health;
- passive health for the main cluster;
- max destination connections;
- activity timeout;
- HTTP version policy;
- response-buffering policy;
- request-size limits before proxying.

Rate limiting

Gateway public limits are stricter than backend limits. Backend limits provide a second layer against bypass, misrouting and internal runaway traffic. Queue defaults to zero; fast 429 rejection during overload reduces memory pressure and tail latency compared with building a large in-memory queue.

Optimizations deliberately not made

- No Redis wallet balance as source of truth.
- No blind weakening of financial SQL isolation.
- No automatic retry for arbitrary financial POSTs.
- No authenticated financial-response caching.

- No unbounded DB/HTTP connection pool.
- No bypassing fraud checks for latency.
- No ledger denormalization that would break reconciliation.

Benchmark plan

Measure:

1. registration RPS;
2. login p50/p95/p99;
3. wallet-list p50/p95/p99;
4. transfer p50/p95/p99;
5. same-wallet concurrent transfer throughput;
6. Gateway overhead versus a controlled direct internal baseline;
7. Redis OTP latency;
8. SQL locks/deadlocks;
9. CPU/memory/GC/socket counts;
10. ledger/balance/idempotency reconciliation after load.

A benchmark result is invalid if final balances and ledger state do not reconcile.